

BANKING SYSTEM

QUESTION :

Fundamentals of Object Oriented Programming:

Object oriented paradigm, Object and Classes, Data Abstraction and Encapsulation, Inheritance, Polymorphism, Dynamic Binding.

1. Sample Case study on class & objects CO1

Bank Account Management System:

Problem Statement: Design a system to manage bank accounts.

Classes: Account, Savings-Account, Current-Account.

Attributes: Account number, balance, account type, interest rate (for savings account), overdraft limit (for current account), etc.

Methods: Deposit, Withdraw, Calculate Interest, Transfer, etc.

Object Usage: Create objects for each customer's account and perform transactions like deposits, withdrawals, and transfers.

EXECUTION STEPS :

1. Open IntelliJ IDEA and select New Project → Java. Select an installed JDK and create the project.
2. Right-click the src folder → New → Java Class and name it Bank.
3. Paste the given Java program into Bank.java. Since the public class is Bank, the file name must be Bank.java.
4. Save the program using Ctrl + S.
5. Click the green Run button next to the main() method, or right-click the Bank class and select Run 'Bank.main()'.
6. Check the Run/Console window at the bottom of IntelliJ IDEA.
7. Verify the transactions: deposit, withdrawal, interest calculation, transfer, overdraft-limit check, and final account details.
8. Take a screenshot of the IntelliJ console if an output screen.

ALGORITHM :

Step 1: Start the program.

Step 2: Create a class `Account` with the data members:

- Account number
- Balance
- Account type

Step 3: Create a parameterized constructor in `Account` to initialize these values.

Step 4: Create methods in `Account` for:

- Displaying account details
- Depositing money
- Withdrawing money
- Transferring money from one account to another

Step 5: Create a class `SavingsAccount` that inherits from `Account`.

Step 6: Add an interest rate to `SavingsAccount` and initialize it using the constructor.

Step 7: Create a method to calculate interest and add the interest to the savings account balance.

Step 8: Create a class `CurrentAccount` that inherits from `Account`.

Step 9: Add an overdraft limit to `CurrentAccount`.

Step 10: Override the `withdraw()` method in `CurrentAccount` to allow withdrawal up to the available balance plus the overdraft limit.

Step 11: In the `main()` method, create objects for `SavingsAccount` and `CurrentAccount`.

Step 12: Display the initial details of both accounts.

Step 13: Deposit money into the savings account.

Step 14: Withdraw money from the current account.

Step 15: Calculate interest for the savings account and update its balance.

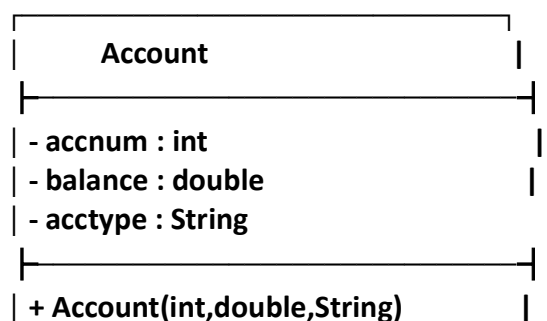
Step 16: Transfer money from the savings account to the current account.

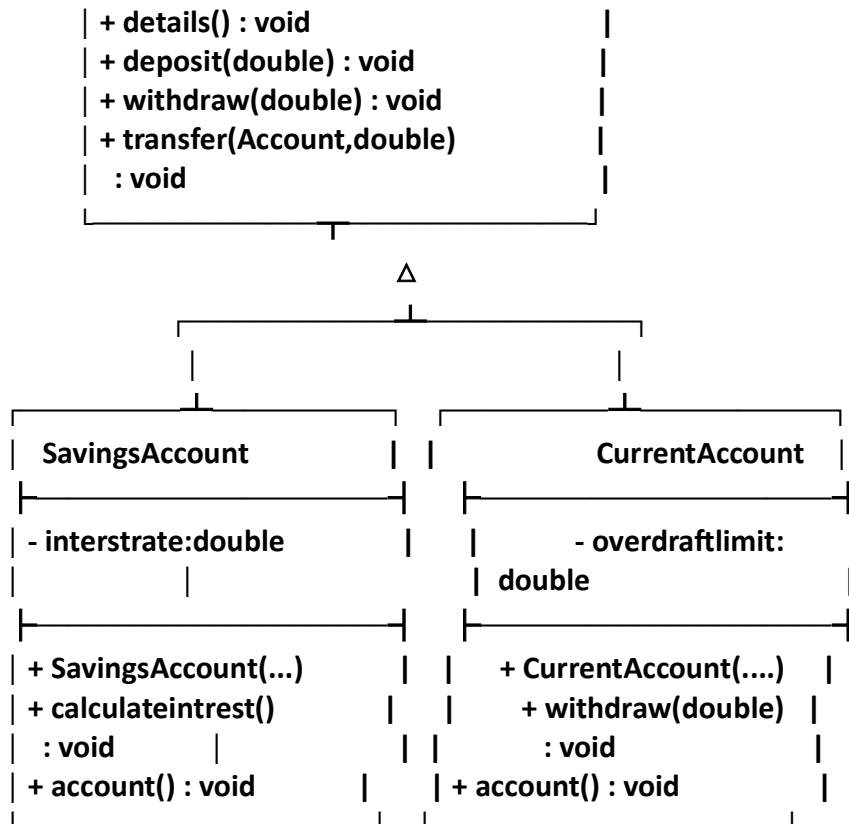
Step 17: Withdraw money from the current account and check the overdraft limit.

Step 18: Display the final details and updated balances of both accounts.

Step 19: Stop the program.

UML DIAGRAM:





CODE :

```

class Account{
    int accnum;
    double balance;
    String acctype;
    public Account(int n,double m,String l){
        this.accnum=n;
        this.balance=m;
        this.acctype=l;
    }
    void details() {
        System.out.println("accnum :"+accnum);
        System.out.println("balance :"+balance);
        System.out.println("acctype :"+acctype);
    }
    void deposit(double amount) {

        balance = balance + amount;
        System.out.println("Deposited: " + amount);
    }
}
  
```

```

    }
    void withdraw(double amount){
        if (amount<=balance) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Insufficient balance");
        }
    }
    void transfer(Account receiver,double amount) {
        if(amount > 0 && amount <= balance) {
            this.balance = this.balance - amount;
            receiver.balance = receiver.balance + amount;
            System.out.println("Transferred: " + amount);
            System.out.println("To Account: " + receiver.accnum);
        }else {
            System.out.println("Transfer failed");
        }
    }
}

class SavingsAccount extends Account{
    double interstrate;
    public SavingsAccount(int n,double m,double k){
        super(n,m,"saving account");
        this.interstrate=k;
    }
    void calculateintrest() {
        double intrest = balance * (interstrate / 100);
        balance = balance + intrest;
        System.out.println("value :"+intrest);
    }
    void account() {
        System.out.println("accnum :"+accnum);
        System.out.println("balance :"+balance);
        System.out.println("interstrate :"+interstrate);
    }
}

class CurrentAccount extends Account{
    double overdraftlimit;
    public CurrentAccount(int n,double m,double o){
        super(n,m,"current account");
        this.overdraftlimit=o;
    }
    @Override

```

```

void withdraw(double amount){
    if (amount > 0 && amount <= balance+overdraftlimit) {
        balance = balance - amount;
        System.out.println("Withdrawn: " + amount);
    } else {
        System.out.println("exceed overdraft limit");
    }
}

void account() {
    System.out.println("accnum :"+accnum);
    System.out.println("balance :"+balance);
    System.out.println("overdraftlimit :"+overdraftlimit);
}

}

public class Bank {
    public static void main(String []args) {
        SavingsAccount s = new SavingsAccount(3000,2424,50);
        CurrentAccount c = new CurrentAccount(2254,5845,300);
        System.out.println("savings account");
        s.account();
        System.out.println("\ncurrent account");
        c.account();
        System.out.println("\nDeposit");
        s.deposit(1000);
        System.out.println("\nwithdraw");
        c.withdraw(2000);
        System.out.println("\ninterstrate");
        s.calculateintrest();
        System.out.println("\ntransfer");
        s.transfer(c,500);
        System.out.println("\nwithdraw");
        c.withdraw(5000);
        System.out.println("\nfinal account details\n");
        s.account();
        c.account();
    }
}

```

OUTPUT :

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.4\lib\idea_rt.jar=60122" -Dfile.encoding=UTF-8 -Dsun
The salary of a person is: 456600
The id of a person is: 222
The name of a person is: manodhara
----- Employee Details -----
The id of a person is: 345
The name of a person is: bhanu
The salary of a person is: 200000
The designation of a person is: false
The promotion of a person is: true

The salary of a person is: 500000
The id of a person is: 300
The name of a person is: manikanta

Process finished with exit code 0
```